DocSense — LLM-Powered Developer Documentation Generator

A Research Collaboration Proposal

Prepared by	[Your Name], Lead Developer
Submitted to	[PhD Researcher Name], [University / Lab Name]
Date	April 2026
Version	1.1 — Updated 20-Day Rapid MVP

---

1. Executive Summary

DocSense is a VS Code extension that addresses one of the most persistent and well-documented problems in software engineering: the systematic avoidance of code documentation. When a developer selects any function, class, or module, DocSense generates clean, structured documentation in the correct format for that language — JSDoc for JavaScript/TypeScript, Docstrings for Python — along with realistic usage examples.

The key differentiator is not documentation generation itself, but **style-adaptive generation**: DocSense analyzes the existing documentation patterns within a project and ensures that all generated output matches the team's established voice, verbosity level, and structural conventions. The result is documentation that is indistinguishable from what the team would have written manually.

> **Core value proposition:** *"DocSense generates documentation that looks like your team wrote it, not like an AI guessed it."*

This proposal outlines the technical architecture, MVP scope, **20-day rapid delivery roadmap**, and the academic research opportunities this platform enables.

---

2. The Problem

Documentation is universally accepted as essential and universally skipped in practice. The root cause is not laziness — it is a structural incentive problem: documentation is time-consuming, delivers no immediate functional value, and its absence is rarely punished until a project scales.

Problem	Real-World Impact
Functions committed with no comments	Onboarding new developers requires reading source code rather than documentation
Documentation becomes stale after code changes	Incorrect documentation is actively harmful — worse than no documentation

Problem	Real-World Impact
Every developer writes docs differently	No consistent voice across a codebase; documentation quality is person-dependent
JSDoc / Docstring syntax is pure repetition	Engineers deprioritize it to maintain shipping velocity
Open-source projects lose contributors	New contributors cannot navigate an undocumented codebase

These are not new problems. They are well-studied in software maintenance literature. What is new is that LLMs in 2025–2026 are accurate enough at constrained text generation tasks — where inputs and outputs are well-defined — to solve this at scale, inside the developer's existing workflow.

---

3. Competitive Landscape

Tool	Their Approach	Limitation
GitHub Copilot	Writes code; adds brief inline comments	Not designed for structured documentation generation
ChatGPT / Claude	Manual prompting; no project context	Requires context-switching; no style awareness
Mintlify	Generates JSDoc for single functions	No style learning, no codebase awareness, no usage examples
DocSense	Style-adaptive, in-editor, preview-gated generation	Uniquely combines project-level style memory with an approval workflow

DocSense is not competing on "AI documentation generation" broadly. It competes on **consistency** — the ability to make AI-generated docs feel like they were written by someone who has been on the team for two years.

---

4. Technical Architecture

4.1 Design Philosophy

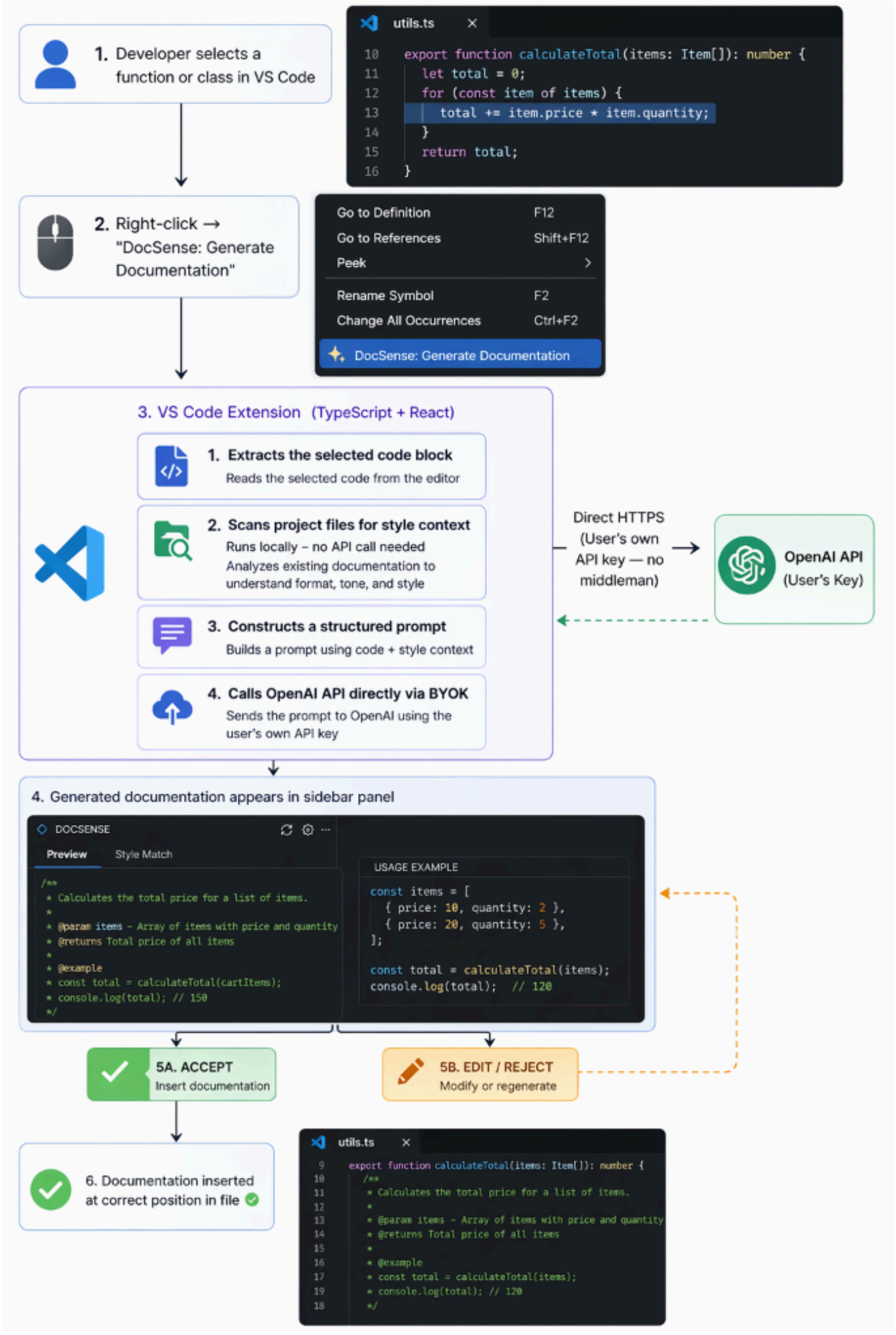
The architecture is designed around three constraints:

1. **Privacy first:** The developer's source code must never leave their own accounts or machines without their explicit knowledge and consent.
2. **Minimal friction:** The tool must integrate into the existing VS Code workflow with a single right-click. No new terminals, no context switching.
3. **Reviewer control:** Generated documentation must always be reviewed before insertion. The system must never modify files automatically.

4.2 System Overview

DocSense follows a **Hybrid Intelligence** model:

- **Local Analysis:** Static analysis (AST) and Style Extraction happen entirely on the local machine.
- **AI Generation:** The final text is generated via LLM using the user's own API key (BYOK).


1. Developer selects a function or class in VS Code
utils.ts
export function calculateTotal(items: Item[]): number {
let total = 0;
for (const item of items) {
total += item.price * item.quantity;
}
return total;
}
2. Right-click → "DocSense: Generate Documentation"
Go to Definition F12
Go to References Shift+F12
Peek
Rename Symbol F2
Change All Occurrences Ctrl+F2
DocSense: Generate Documentation
3. VS Code Extension (TypeScript + React)
1. Extracts the selected code block
Reads the selected code from the editor
2. Scans project files for style context
Runs locally – no API call needed
Analyzes existing documentation to understand format, tone, and style
3. Constructs a structured prompt
Builds a prompt using code + style context
4. Calls OpenAI API directly via BYOK
Sends the prompt to OpenAI using the user's own API key
Direct HTTPS (User's own API key — no middleman)
OpenAI API (User's Key)
4. Generated documentation appears in sidebar panel
DOCSENSE
Preview
Style Match
/**
* Calculates the total price for a list of items.
*
* @param items - Array of items with price and quantity
* @returns Total price of all items
*
* @example
* const total = calculateTotal(cartItems);
* console.log(total); // 150
*/
USAGE EXAMPLE
const items = [
{ price: 10, quantity: 2 },
{ price: 20, quantity: 5 },
];
const total = calculateTotal(items);
console.log(total); // 120
5A. ACCEPT
Insert documentation
5B. EDIT / REJECT
Modify or regenerate
6. Documentation inserted at correct position in file
utils.ts
export function calculateTotal(items: Item[]): number {
/**
* Calculates the total price for a list of items.
*
* @param items - Array of items with price and quantity
* @returns Total price of all items
*
* @example
* const total = calculateTotal(items);
* console.log(total); // 120
*/

4.3 Privacy Architecture

The MVP uses a **Bring-Your-Own-Key (BYOK)** model with a direct API call:

- The extension communicates with the OpenAI API using the **developer's own API key**, stored locally in VS Code settings.
- No DocSense-controlled server exists in the request chain for the MVP.
- Source code is transmitted only to the developer's own OpenAI account, subject to OpenAI's own data policies.
- The local style context scan is performed **entirely within the extension** — no code leaves the machine for this step.

---

5. Core MVP Features — 20-Day Scope

#	Feature	Description
1	**Inline Documentation Generator**	Right-click any function or class → generate format-correct documentation
2	**Style Detection Engine**	Analyzes existing project docs to match tone, structure, and verbosity
3	**Preview & Accept Sidebar**	All generated docs are shown for review before any file is modified
4	**Usage Example Generation**	Produces a realistic call-site example for each documented function
5	**Multi-Language Support**	Python (Docstrings) and JavaScript / TypeScript (JSDoc)
6	**BYOK Configuration**	Developer enters their own OpenAI API key — no shared credentials

Out of Scope for MVP

The following are explicitly deferred to maintain 20-day delivery focus:

- Git commit-aware documentation changelog generation
- README generation from project structure
- Offline / local model support (e.g., Ollama)

---

6. Research & Academic Contributions

DocSense is designed from the ground up as a research platform.

Study 1 — Style-Adaptive Documentation Generation
Research Question: Can LLMs reliably learn and replicate team-specific documentation styles from a small number of in-context examples?

Study 2 — Code-to-Documentation Accuracy Benchmark
Contribution: Creating a labeled dataset pairing raw function signatures with human-written and LLM-generated documentation across multiple styles.

Study 3 — Developer Trust in AI-Generated Documentation
Research Question: What factors determine whether a developer accepts, edits, or rejects AI-generated documentation?

---

7. 20-Day Rapid MVP Delivery Roadmap

Phase	Days	Deliverable	Verification Checkpoint
Phase 1: Foundation	1–6	Extension scaffold, sidebar UI, and basic Style Scanner	Extension activates and correctly identifies project documentation style
Phase 2: Core Loop	7–13	LLM integration (BYOK), "Accept/Edit" workflow, and File Insertion	Selection -> Generate -> Review -> Insert loop is fully functional
Phase 3: Hardening	14–20	Multi-language support (Python/TS), edge-case testing, and packaging	Stable `.vsix` ready for installation; demo video and MVP report complete

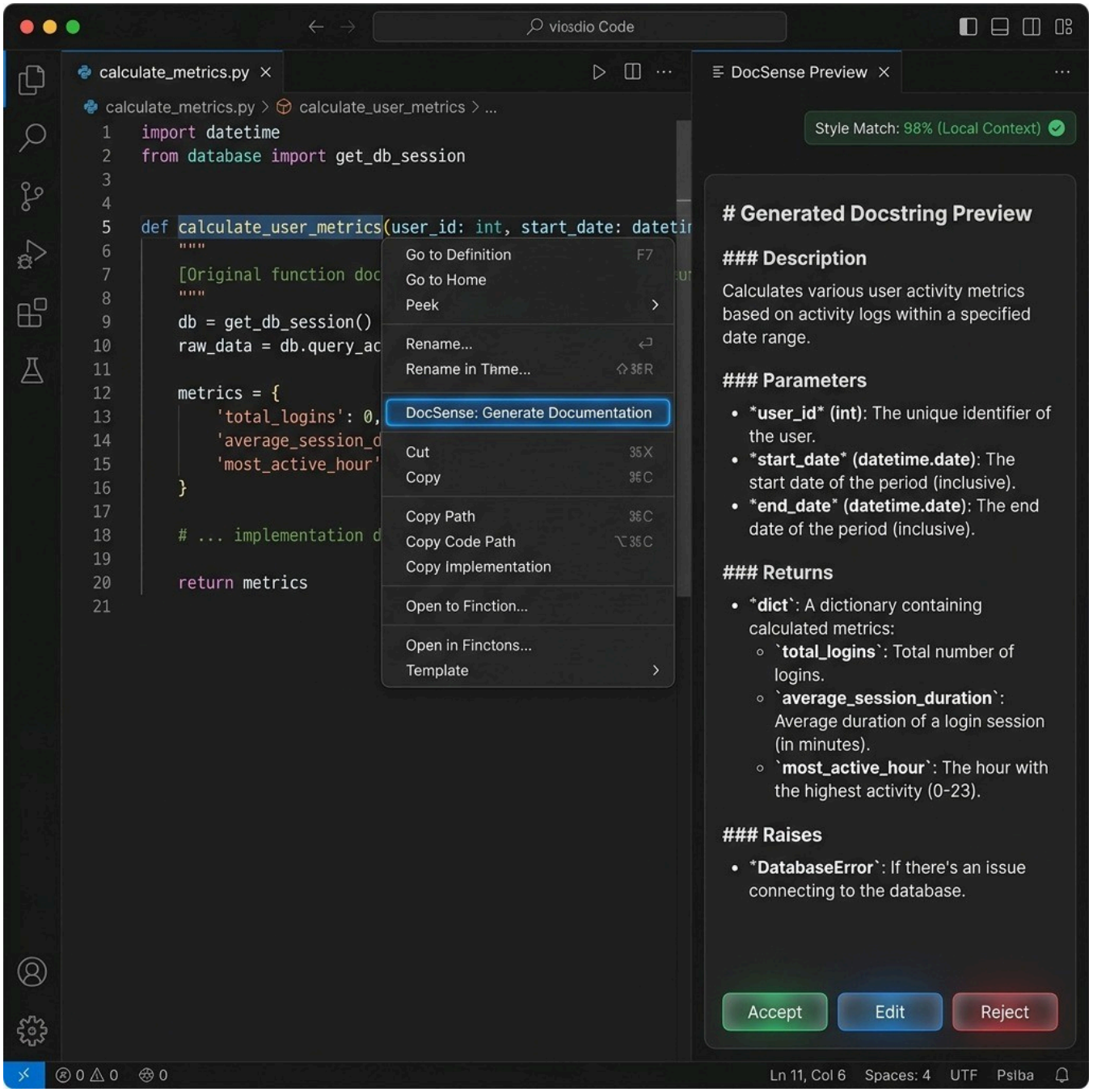
---

8. Success Metrics

Metric	Target
Beta users engaged by Day 20	10 developers
Total documentation blocks generated	200+
Acceptance rate (inserted without edits)	≥ 50% of generated docs

---

9. Closing Statement



DocSense is a focused, technically disciplined tool addressing a problem that every software team has and no existing product solves well. The 20-day MVP scope is deliberately constrained to deliver a working, privacy-respecting, installable extension that validates the core hypothesis: **style-adaptive LLM documentation generation is useful, trusted, and measurably faster than manual documentation.**

DocSense — LLM-Powered Developer Documentation Generator
Updated Research Collaboration Proposal — April 2026